講義 10：Report Event（報表事件：系統在固定時機執行的程式區塊）——事件流程與互動清單（Interactive List）（授課順序：接在講義 15 之後）

> 對應練習：ex10 | 答案程式：`ZR_TR10_EVENTS`

本講重點

- Report 程式的真相：**不是從上跑到下，而是事件驅動**
- 事件時序：INITIALIZATION → AT SELECTION-SCREEN → START-OF-SELECTION → END-OF-SELECTION
- 清單事件：TOP-OF-PAGE / END-OF-PAGE
- **清單緩衝區（List Buffer）**：WRITE 先寫進記憶體，事件跑完才顯示
- 互動清單：AT LINE-SELECTION 與 **HIDE**（隱藏區、寫回時機、使用規則）、`sy-lsind` / `sy-lilli` / `sy-lisel`
- `MESSAGE` 訊息類型

1. 事件驅動：位置無關，時機決定

Report 程式由一塊塊「事件區塊」組成：事件關鍵字出現到下一個事件關鍵字（或 FORM 區）之前的程式碼屬於該事件，**由系統在對應時機呼叫**，跟寫在檔案裡的先後順序完全無關。你可以把 TOP-OF-PAGE 寫在最後面，它照樣在每頁開頭執行。

> 隱含規則：程式開頭第一段「不屬於任何事件」的可執行程式碼，隱含屬於 START-OF-SELECTION。初學建議一律明寫事件關鍵字，不靠隱含。

2. 事件時序總表

順序	事件	觸發時機	典型用途
1	`LOAD-OF-PROGRAM`	程式載入（最早）	少用，先認識
2	`INITIALIZATION`	選擇畫面（Selection Screen）顯示**之前**	給選擇畫面預設值、框標題
3	`AT SELECTION-SCREEN OUTPUT`	選擇畫面每次顯示前	動態調整畫面（進階）
4	`AT SELECTION-SCREEN`	使用者按執行後、主邏輯**之前**	**驗證輸入**，擋下錯誤條件
5	`START-OF-SELECTION`	驗證通過後	**主處理**：取數、加工、輸出
6	`END-OF-SELECTION`	主處理結束	總結、合計、（期末）總頁數回填
-	`TOP-OF-PAGE`	基本清單每頁開頭（由該頁第一個 WRITE 觸發）	頁首（Page Header）、表頭

順序	事件	觸發時機	典型用途
-	END-OF-PAGE	每頁到達保留行區（需 LINE-COUNT n(m)）	頁尾（Page Footer）
-	AT LINE-SELECTION	使用者雙擊清單行（或 F2）	互動明細（第二層清單）

3. 各事件重點與範例

3.1 INITIALIZATION：預設值

```
INITIALIZATION.
  t_b1 = '查詢條件'.                  " BLOCK 框標題
  gs_score-sign   = 'I'.             " 給 SELECT-OPTIONS 預設範圍 0~100
  gs_score-option = 'BT'.
  gs_score-low    = 0.
  gs_score-high   = 100.
  APPEND gs_score TO s_score.
```

3.2 AT SELECTION-SCREEN：輸入驗證

```
AT SELECTION-SCREEN.
  LOOP AT s_score INTO gs_score.
    IF gs_score-low < 0 OR gs_score-high > 999.
      MESSAGE '成績範圍請輸入 0～999' TYPE 'E'.
    ENDIF.
  ENDLOOP.
```

`MESSAGE ... TYPE 'E'` 在這個事件裡的效果：顯示錯誤、**把使用者留在選擇畫面**，改對了才放行。訊息類型速覽：

TYPE	效果
I	彈窗訊息，按確定繼續
S	狀態列訊息，不中斷
W	警告（選擇畫面上可按 Enter 硬過）
E	錯誤：擋在選擇畫面
A	中止：程式直接結束

3.3 START-OF-SELECTION / END-OF-SELECTION

```
START-OF-SELECTION.
  PERFORM get_data.
  PERFORM display_data.
```

```
END-OF-SELECTION.
  ULINE.
  WRITE: / '符合條件筆數：', gv_count.
```

查無資料時提早結束：`STOP`。舊程式常這樣寫（講義 13 的實戰案例 ZRFI0004）：

```
START-OF-SELECTION.
  SELECT ... INTO TABLE t_dochd ...
  IF t_dochd[] IS INITIAL.
    MESSAGE i001(fot_b2a).      " 彈窗告知查無資料
    STOP.
  ENDIF.
```

注意 `STOP` 不是「整支程式直接結束」：官方文件（`STOP`）規定，它會離開目前的 `START-OF-SELECTION`（或 `AT SELECTION-SCREEN`）區塊，然後**照樣觸發 `END-OF-SELECTION`**。所以 END-OF-SELECTION 裡的輸出邏輯仍然會跑，要能承受「內表（Internal Table）是空的」這個狀況（ZRFI0004 的 `LOOP AT t_dochd` 遇到空表就什麼都不印，剛好沒問題）。`STOP` 只能用在報表程式的這幾個事件，寫在 FORM 裡時要確定它是從這些事件呼叫的。

3.4 TOP-OF-PAGE：每頁表頭

```
TOP-OF-PAGE.
  WRITE: / '學生成績清單（雙擊任一行看明細）'.
  ULINE.
```

注意觸發者是「**輸出**」：該頁第一個 WRITE 執行時系統先跑 TOP-OF-PAGE。一行都沒輸出就不會有頁首。（分頁與 END-OF-PAGE 的細節在講義 12。）

3.5 清單緩衝區（List Buffer）：WRITE 其實先寫進記憶體

它是什麼：程式裡每一個 `WRITE`（還有 `ULINE`、`SKIP`），都不是一行一行即時畫到螢幕上，而是先寫進記憶體裡的**清單緩衝區（List Buffer）**。畫面上的清單，是系統在適當時機從緩衝區一次顯示出來的。

什麼時候顯示：

清單	寫進緩衝區的時機	顯示到畫面的時機
基本清單（`sy-lsind = 0`）	`START-OF-SELECTION`、`END-OF-SELECTION`、`TOP-OF-PAGE` 裡的 `WRITE`	這些事件**全部跑完**之後
明細清單（`sy-lsind = 1、2…`）	`AT LINE-SELECTION` 裡的 `WRITE`	這個事件**跑完**之後

所以程式跑到一半（例如還在 `LOOP` 裡），畫面上還看不到已經 `WRITE` 的內容，要等事件跑完、清單顯示出來才看得到。

裡面有什麼：

- 每一行輸出的**文字**
- **分頁資訊**：每一行在第幾頁、第幾行（講義 12 的 `LINE-COUNT` 決定一頁幾行）
- **隱藏區（Hide Area）**：每一行用 `HIDE` 記下的變數值（第 4 節）

每一層清單各一份：基本清單有一份緩衝區；雙擊產生的每一層明細清單，各自再建一份新的，用 `sy-lsind` 區分。按返回鍵（F3）退回上一層時，上一層的緩衝區還在，畫面直接從它重新顯示，不會重跑程式。

能拿它做什麼：正因為輸出還在記憶體裡、還沒上螢幕，程式可以「回頭」處理已經寫好的行：

做法	用途	在哪一講
`HIDE`	替每一行記下鍵值，雙擊時寫回變數	本講第 4 節
`READ LINE`	把某一行的文字讀回 `sy-lisel`（同時也會把那一行 `HIDE` 過的值寫回變數）	講義 13
`MODIFY LINE`	把改好的 `sy-lisel` 寫回同一行	講義 13（總頁數回填：全部印完才知道總頁數，再回頭改每頁頁首）

4. 互動清單（Interactive List）：AT LINE-SELECTION 與 HIDE

需求：清單只顯示摘要，使用者**雙擊某行**，就跳到下一層清單看那筆資料的明細。

4.1 問題：畫面上的清單只是「文字」

`WRITE` 輸出到畫面後，清單就只是一行一行的文字，**跟程式裡的 internal table 已經沒有關係**。使用者雙擊第 3 行時，系統只知道「第 3 行被點了」，並不知道這一行是哪一位學生——程式裡的 `gs_student` 此時放的是迴圈跑完後的**最後一筆**，不是被點的那一筆。

所以輸出清單時，要替每一行**另外記下**「這一行代表哪一筆資料」的鍵值。這就是 `HIDE` 的工作。

4.2 HIDE 的語法與意義

```
HIDE 變數.
HIDE: 變數1, 變數2.          " 可以同時記多個變數
```

- **意義**：把「變數**當下**的值」跟「**目前這一行清單**」綁在一起，存進系統替這一層清單準備的**隱藏區（Hide Area）**。畫面上看不到，所以叫 HIDE。
- **存在哪裡**：每一層清單各有自己的隱藏區，每一行可以存好幾個變數的值。
- **什麼時候取回**：使用者在清單上雙擊（或 F2）某一行時，系統在觸發 `AT LINE-SELECTION` **之前**，先把**那一行**存過的值，一一**寫回原本的變數**。

隱藏區（Hide Area）是什麼

第 3.5 節提過：`WRITE` 的輸出先放在**清單緩衝區（List Buffer）**。跟 HIDE 有關的是緩衝區裡的兩部分：

	內容	看得到嗎
清單內容	每一行 `WRITE` 出來的文字	看得到，就是畫面上的清單

	內容	看得到嗎
隱藏區	每一行用 HIDE 記下的「變數名稱 + 當時的值」	**看不到**

兩者都依**行號**對應：第 3 行的文字，對應隱藏區裡第 3 行存的值。官方文件的定義是：清單緩衝區中的一個區域，可以替畫面清單的每一行存放全域變數的值。

可以想成**每一行背面貼了一張便利貼**：

- WRITE 把文字寫在正面，給使用者看。
- HIDE gs_student-id 在同一行的背面貼一張便利貼，寫上「gs_student-id = S0002」。
- 使用者雙擊這一行 → 系統翻到背面，照便利貼把 'S0002' 寫回變數 gs_student-id → 再執行 AT LINE-SELECTION。
- 頁首、分隔線沒有 HIDE，背面沒有便利貼，雙擊時什麼都不會寫回。

每一層清單各有一份：基本清單（sy-lsind = 0）有自己的清單緩衝區與隱藏區；雙擊後產生的明細清單（第 1 層）是另一份新的，也有自己的隱藏區。在明細清單裡再 HIDE，存的是明細那一層的資料，不會蓋掉基本清單的；按返回鍵回到上一層時，上一層的隱藏區還在，照樣可以再雙擊別的行。

用本講的學生清單來看，執行完輸出迴圈後，隱藏區長這樣：

清單行（畫面上看得到的）	隱藏區（看不到的）
S0001 王小明 78 91	gs_student-id = 'S0001'
S0002 李小美 88 95	gs_student-id = 'S0002'
S0003 陳大文 60 72	gs_student-id = 'S0003'
頁首、ULINE、合計行	（沒有 HIDE，什麼都沒存）

使用者雙擊 S0002 那一行 → 系統把 'S0002' 寫回 gs_student-id → 執行 AT LINE-SELECTION → 程式用 gs_student-id 去 READ TABLE 找到李小美的完整資料。

4.3 完整範例

```
* 輸出時：每印一行，就把這一行的學號記進隱藏區
LOOP AT gt_students INTO gs_student WHERE score1 IN s_score.
  WRITE: / gs_student-id, gs_student-name, gs_student-score1.
  HIDE gs_student-id.                " 緊接在 WRITE 之後：記住「這一行 = 這個學號」
ENDLOOP.
CLEAR gs_student-id.                 " 防呆：清掉迴圈留下的最後一筆（原因見 4.4 第 4 點）

* 雙擊時：系統已經先把該行 HIDE 過的學號寫回 gs_student-id
AT LINE-SELECTION.
  IF gs_student-id IS INITIAL.       " 點到沒有 HIDE 的行（頁首、分隔線、合計行）
    WRITE / '請雙擊資料行'.
  ELSE.
    READ TABLE gt_students INTO gs_detail WITH KEY id = gs_student-id.
    IF sy-subrc = 0.
```

```
      WRITE: / '=== 學生明細（第', sy-lsind, '層清單）===',
             / '學號：', gs_detail-id,
             / '姓名：', gs_detail-name.
    ENDIF.
  ENDIF.
```

`AT LINE-SELECTION` 裡的 `WRITE` 不會印在原本的清單上，而是產生**新的一層清單**（明細清單），蓋在原清單上面顯示；按返回鍵（F3）回到上一層。

4.4 使用規則（寫錯會跑錯資料或當機）

1. **`HIDE` 要緊接在 `WRITE` 之後**：`HIDE` 綁的是「目前游標所在的那一行」。寫在 `WRITE` 之前，綁到的是上一行；寫在 `WRITE: /` 之後，綁到的才是剛印出來的這一行。
2. **只能 HIDE 全域變數**：程式開頭（或 TOP include）宣告的變數才行。在 FORM 裡 HIDE 一個 FORM 自己的區域變數（`DATA` 宣告在 FORM 裡的），執行時直接 dump（Runtime Error `HIDE_NO_LOCAL`）——因為 FORM 結束後區域變數就消失了，系統之後沒有地方可以寫回。
3. **只能 HIDE 平面（flat）型別**：單一欄位、或全部由單一欄位組成的結構都可以；internal table、`string` 這類長度會變的型別不行，也不要 HIDE Field-Symbol。實務上只 HIDE **鍵值**（如學號、訂單號），明細資料在 `AT LINE-SELECTION` 裡再用鍵值去查。
4. **只有 HIDE 過的變數會被寫回，其他變數維持原值**：雙擊一行沒有 HIDE 的行（頁首、`ULINE`、合計行），系統什麼都不會寫回，`gs_student-id` 仍然是迴圈跑完留下的**最後一筆**——看起來像點了最後一位學生。所以輸出完要 `CLEAR`，事件裡再用 `IS INITIAL` 判斷有沒有點到資料行。
5. **HIDE 的鍵值要足以唯一識別一筆資料**：例如航班要 HIDE `carrid`、`connid`、`fldate` 三個欄位（`HIDE: gs_flight-carrid, gs_flight-connid, gs_flight-fldate.`），只 HIDE `carrid` 就分不出是哪一班。

> **實測（2026-09-25，驗證程式 `ZR_TR10_HIDE_TEST`）**：同一行在 `WRITE` 之前 HIDE 一個變數（`gv_before`）、之後 HIDE 另一個（`gv_after`），依序雙擊三行的結果：
>
> | 雙擊哪一行 | `gv_after`（**WRITE 之後**） | `gv_before`（**WRITE 之前**） |
> |---|---|---|
> | S0001 | S0001 | **S0002** |
> | S0002 | S0002 | **S0003** |
> | S0003 | S0003 | **S0003**（上一次雙擊殘留的值） |
>
> - 第 1 條：寫在 `WRITE` 之前的 HIDE，綁到的是上一行——所以 S0001 那一行取回的是「下一筆」S0002。
> - 第 4 條：S0003 那一行沒有 `gv_before` 的 HIDE，雙擊時系統不寫回它，`gv_before` 保留**上一次雙擊 S0002 時寫回的 S0003**。殘留的不只是迴圈最後一筆，連前一次雙擊的值都會留著。

4.5 多層清單與相關系統欄位（System Field）

在 `AT LINE-SELECTION` 裡輸出明細時**再 HIDE 一次**，使用者就可以在明細清單上再雙擊，往下鑽到第三層——每一層清單都有自己獨立的隱藏區，互不干擾。

系統欄位	意義
sy-lsind	目前正在產生的清單層級：基本清單 0，第一層明細 1，依此類推（最多 20 層）
sy-lilli	被雙擊的那一行，是清單的第幾行
sy-lisel	被雙擊那一行的**畫面文字內容**（整行字串）

> 為什麼不用 sy-lisel 從畫面文字裡切出學號？因為畫面文字的欄位位置會隨輸出格式改變，切字串很脆弱；HIDE 直接存變數的值，跟畫面怎麼排版無關，是標準做法。

4.6 延伸：雙擊後串到另一支報表

明細如果已經有現成的報表，AT LINE-SELECTION 裡可以不自己 WRITE，改成呼叫那支報表，並把 HIDE 取回的鍵值當選擇條件（Selection Criteria）傳過去：

```
AT LINE-SELECTION.
  IF gs_student-id IS NOT INITIAL.
    SUBMIT zr_student_detail                    " 另一支報表
      WITH p_id = gs_student-id                 " 對方選擇畫面上的 PARAMETERS p_id
      AND RETURN.                               " 對方結束後回到本清單
  ENDIF.
```

- WITH p_id = ...：把值帶進對方選擇畫面的 PARAMETERS p_id。沒有加 VIA SELECTION-SCREEN 時，**對方的選擇畫面會被跳過**，直接執行並顯示清單。
- AND RETURN：使用者看完對方的報表按 F3，會回到原本的清單；不加的話，本程式會直接結束。

> **實測（2026-09-25）**：驗證程式 ZR_TR10_HIDE_TEST 在明細清單雙擊後 SUBMIT zr_tr10_hide_detail WITH p_id = gv_after AND RETURN：畫面直接顯示 ZR_TR10_HIDE_DETAIL 的清單，沒有出現它的 P_ID 選擇畫面，收到的 p_id 就是 HIDE 取回的 S0002；按 F3 回到 ZR_TR10_HIDE_TEST 的明細清單，程式沒有結束。兩支程式的原始碼見 zr_tr10_hide_test.prog.abap / zr_tr10_hide_detail.prog.abap，可直接在 SE38 執行體驗。不管是自己印下一層清單、還是串到另一支報表，**關鍵都一樣：靠 HIDE 知道使用者點的是哪一筆**。

5. 常見錯誤與陷阱

症狀	原因
INITIALIZATION 的預設值沒出現	事件名拼錯（被當成普通程式碼歸入 START-OF-SELECTION）
驗證訊息跳完程式照跑	TYPE 用了 I/S——擋人要用 E
雙擊任何行都顯示同一筆	忘了在輸出迴圈裡 HIDE，變數殘留最後一筆
雙擊後顯示的是「上一行」的資料	HIDE 寫在 WRITE 之前，綁到了上一行
執行時 dump HIDE_NO_LOCAL	在 FORM 裡 HIDE 了 FORM 的區域變數；要 HIDE 全域變數
航班明細查到別的日期	HIDE 的鍵值不完整（只 HIDE carrid），要把能唯一識別的欄位都 HIDE

症狀	原因
雙擊空白行出現殘留資料	輸出後沒 CLEAR + 事件裡沒檢查 IS INITIAL
TOP-OF-PAGE 沒執行	該頁沒有任何 WRITE 輸出
程式碼寫在事件關鍵字之前	隱含屬於 START-OF-SELECTION，時序跟預期不同

6. 課堂練習

完成 ex10：完整走一遍六個事件——預設範圍、輸入驗證、主處理、總結、頁首、雙擊明細（HIDE + sy-lsind）。